

[32] Vespa: AI + data, online at any scale 총정리

개발사: Yahoo

형태: 대규모 AI + 데이터 서빙 플랫폼

웹사이트: <https://vespa.ai/>

공개/업데이트: 2024

요약

Vespa 는 Yahoo 에서 개발한 오픈소스 검색 및 AI 데이터 서빙 플랫폼이다. 단순한 벡터 DB 를 넘어, 구조화된 데이터, 벡터, 그리고 복잡한 추론 로직을 모두 포괄하는 포괄적 시스템이다. Vespa 의 핵심 특징은 실시간 데이터 업데이트와 높은 처리량의 개인화된 검색/추천을 동시에 제공한다는 점이다. 대규모 인터넷 서비스에서 사용되었던 검색 및 추천 엔진의 경험을 바탕으로, 프로덕션급 신뢰성과 확장성을 갖추고 있다. 텍스트 검색, 벡터 검색, 정형 데이터 필터링, 그리고 머신러닝 모델의 추론을 하나의 통일된 플랫폼에서 처리하므로, 현대적 AI 애플리케이션에 매우 적합하다.

상세분석

32.1 주요 문제점과 Vespa 의 해결책

기존 검색/추천 시스템의 한계:

- **오프라인-온라인 갭:** 오프라인에서 학습한 모델을 온라인 시스템에 배포할 때의 복잡성과 지연
- **실시간 개인화의 어려움:** 사용자의 최근 행동을 반영한 즉시 개인화 추천의 기술적 복잡성
- **다양한 데이터 타입 통합:** 정형 데이터, 텍스트, 벡터, 그래프 등을 통일된 방식으로 처리
- **확장성과 지연시간:** 초당 수백만 요청을 수백 밀리초 이내에 처리
- **AI 모델 배포:** 새로운 ML 모델의 빠른 반영과 버전 관리

Vespa 의 해결책:

- 통일된 플랫폼으로 모든 데이터와 로직을 처리
- 실시간 데이터 피드 처리
- 쿼리 시 동적 추론 실행
- 자동 확장과 고가용성

32.2 핵심 아키텍처

계층적 구조

```

클라이언트 애플리케이션
↓
Query API (REST, gRPC)
↓
Dispatch 계층 (로드 밸런싱)
↓
Search 노드 (인덱싱, 검색)
↓
저장소 (MVCC 기반 인덱스)
↓
Content 저장소 (Data 노드)

```

주요 구성 요소

1. Content Layer:

- 실제 문서 데이터 저장
- 분산 파티셔닝
- 자동 복제

2. Search Layer:

- 역인덱스 (Inverted Index) 기반 검색
- 벡터 유사도 검색
- 랭킹 및 재정렬

3. Query Language (YQL):

- 강력한 쿼리 언어
- 필터, 정렬, 그룹화, 함수 지원

32.3 주요 기능

1. 다양한 검색 기능

텍스트 검색:

- 전문 검색 (Full-text Search)
- 구문 검색 (Phrase Search)
- 부분 일치 (Partial Matching)
- 언어별 형태소 분석

```
select * from documents
where default contains "AI" and title contains "machine learning"
```

벡터 검색:

- 근사 최근접 이웃 (ANN)
- 하이브리드 텍스트-벡터 검색
- 다양한 거리 메트릭

```
select * from documents
where (all(field:embedding, f(x)(x-q)))
order by closeness(field, embedding)
```

메타데이터 필터링:

- 범위 필터
- 카테고리 필터
- 복합 불린 조건

```
select * from documents
where category = "news" and publication_date > 2024-01-01
```

2. 랭킹 및 개인화

다단계 랭킹 (Multi-stage Ranking):

```

1 단계: 매칭 (Matching) - 조건을 만족하는 문서 선별
↓
2 단계: 1 차 랭킹 (First-phase Ranking) - 빠른 점수 계산
↓
3 단계: 2 차 랭킹 (Second-phase Ranking) - 복잡한 점수 계산
↓
4 단계: 재정렬 (Reranking) - 최종 순서 결정

```

개인화 점수:

- 사용자 특성 기반 가중치 조정
- 실시간 피드백 반영

점수(doc, user) = 기본_점수(doc) × 개인화_승수(user, doc)

3. 머신러닝 모델 배포

온라인 추론:

- 쿼리 시점에 ML 모델 실행
- ONNX, TensorFlow 모델 지원

```

select * from documents
where (all(field:embedding, f(x)(x-q)))
order by ml_score(ml_model_v2, field:features)

```

모델 버전 관리:

- A/B 테스트 지원
- 점진적 롤아웃
- 빠른 롤백

4. 실시간 데이터 업데이트

피드 처리 (Feed Processing):

- 클라이언트로부터 문서 스트림 수신
- 파싱, 변환, 검증
- 분산 저장소에 기록
- 인덱스 자동 갱신

```
Document doc = new Document("id:namespace:doctype::1",
    new StringFieldValue("content"));
doc.setFieldValue("embedding", vectorValue);
feedClient.put(doc);
```

일관성 보장:

- 쓰기-읽기 일관성 (Read-after-write consistency)
- 분산 노드 간 동기화

5. 복합 쿼리 처리

YQL (Vespa Query Language):

```
select * from documents
where (all(field:embedding, f(x)(x-q))
    or title contains @text)
and category in ("news", "blog")
and publication_date > now() - 30 days
order by nativeRank, closeness(field, embedding)
limit 10
offset 0
```

32.4 성능 특성

확장성

수평 확장:

- 문서 수에 따른 자동 샤딩
- 노드 추가로 선형 성능 향상
- 지연시간 유지

처리량:

- 초당 수백만 쿼리
- 초당 수십만 데이터 업데이트

지연시간

P99 지연시간: 일반적으로 10-50ms

- 텍스트 검색: 10-20ms
- 벡터 검색: 20-50ms
- 복합 쿼리: 50-100ms

저장소 효율성

압축 기법:

- 비트맵 압축
- 휴프만 인코딩
- 양자화 (벡터용)

메모리 사용:

- 인덱스: 원본 데이터의 10-30%
- 벡터: 압축으로 4-10 배 감소 가능

32.5 배포 모델

클라우드 관리형

Vespa Cloud:

- 완전 관리형 호스팅
- 자동 확장, 고가용성
- 빠른 배포

자관리형

On-Premise 또는 자체 클라우드:

- 완전 제어
- 커스터마이징 자유도
- 운영 책임

32.6 사용 사례

온라인 추천 시스템:

- 사용자 행동 실시간 반영
- 개인화된 상품/콘텐츠 추천
- 대규모 동시 사용자 처리

검색 엔진:

- 웹 검색
- 전자상거래 검색
- 엔터프라이즈 검색

광고 타겟팅:

- 실시간 입찰 (RTB)
- 사용자-광고 매칭

32.7 본 논문과의 관계

Exqutor 은 텍스트 쿼리를 벡터 기반 검색으로 변환하는 하이브리드 검색 시스템이다. Vespa 는 다음의 측면에서 Exqutor 과 관련:

1. **포괄적 플랫폼:** Exqutor 이 구현해야 하는 텍스트-벡터 하이브리드 검색의 완성된 프로덕션 예시를 제공
 2. **다단계 랭킹:** Exqutor 의 검색 결과 순위 지정 전략에 영감 제공
 3. **실시간 업데이트:** Exqutor 이 동적 데이터셋에서 운영되는 경우의 아키텍처 참고
 4. **확장성 경험:** 대규모 분산 환경에서의 벡터 검색 최적화 전략
 5. **AI 모델 통합:** Exqutor 과 외부 임베딩 모델 또는 재순위 모델의 통합 방식
-

추가 제기 문제

1. **다단계 랭킹의 비용-정확도 분석:** 더 복잡한 2 차 랭킹은 더 정확하나 느리다. 최적의 단계 수와 복잡도는?
2. **벡터 검색의 정확도 보장:** 대규모 벡터 인덱스에서 Recall 이 어느 수준 이상 유지되는가? 인덱스 크기에 따른 저하 정도는?
3. **실시간 모델 배포:** 새 ML 모델을 배포할 때 기존 사용자 요청에 미치는 영향 최소화 방안은?
4. **메모리 vs 정확도:** 벡터 압축으로 메모리를 절감할 때, 검색 정확도 손실은 얼마나 되는가?
5. **피드 레이턴시:** 데이터 업데이트 후 검색 결과에 반영되는 시간은? 일관성 vs 성능 트레이드오프는?
6. **모듈식 확장성:** 사용자 정의 검색 로직이나 재순위 함수를 추가할 때의 복잡도는?